except that in this case, `Robot` is a class already implemented by Rob Tuse:

```
class Robot
  constructor: (@at=0) ->
  position: ->
    @at
  move: (displacement) ->
    @at += displacement
  startEngine: -> console.log 'start engine'
  stopEngine: -> console.log 'stop engine'
  forward: ->
    @startEngine()
    @move 1
    @stopEngine()
  reverse: ->
    @startEngine()
    @move -1
    @stopEngine()
```

You can see that that's repetitive. Instead of repeating the same code at the start and at the end of all the methods, you could use the existing `around` function to wrap all of the methods. Unfortunately, everything breaks when you change the `Robot` class to use the `around` defined so far:

```
class Robot
  withRunningEngine = around (action) ->
    @startEngine()
    action()
    @stopEngine()
  constructor: (@at=0) ->
  position: ->
    @at
  move: (displacement) ->
    console.log 'move'
    @at += displacement
  startEngine: -> console.log 'start engine'
  stopEngine: -> console.log 'stop engine'
  forward: withRunningEngine ->
    @move 1
  reverse: withRunningEngine ->
    @move -1

bender = new Robot
bender.forward()
bender.forward()
# TypeError: Object #<Object> has no method 'startEngine'
```